

Projekt

Peter Bartoš, Šimon Freivald

2025-05-03

Libraries

```
library(tidyverse)      # data science/modeling
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(magrittr)       # pipes
```

```
##
## Attaching package: 'magrittr'
##
## The following object is masked from 'package:purrr':
##
##   set_names
##
## The following object is masked from 'package:tidyr':
##
##   extract
```

```
library(ggplot2)        # graphs
library(patchwork)       # combine graphs
```

```
## Warning: package 'patchwork' was built under R version 4.4.3
```

```
library(corrplot)       # correlation plots
```

```
## Warning: package 'corrplot' was built under R version 4.4.3
## corrplot 0.95 loaded
```

```
library(tidymodels)     # for models
```

```
## Warning: package 'tidymodels' was built under R version 4.4.3
```

```
## -- Attaching packages ----- tidymodels 1.3.0 --
## v broom      1.0.7      v rsample    1.3.0
## v dials      1.4.0      v tune       1.3.0
```

```

## v infer          1.0.7      v workflows      1.2.0
## v modeldata      1.4.0      v workflowsets 1.1.0
## v parsnip        1.3.1      v yardstick   1.3.2
## v recipes        1.2.1

## Warning: package 'dials' was built under R version 4.4.3
## Warning: package 'infer' was built under R version 4.4.3
## Warning: package 'modeldata' was built under R version 4.4.3
## Warning: package 'parsnip' was built under R version 4.4.3
## Warning: package 'recipes' was built under R version 4.4.3
## Warning: package 'rsample' was built under R version 4.4.3
## Warning: package 'tune' was built under R version 4.4.3
## Warning: package 'workflows' was built under R version 4.4.3
## Warning: package 'workflowsets' was built under R version 4.4.3
## Warning: package 'yardstick' was built under R version 4.4.3

## -- Conflicts ----- tidymodels_conflicts() --
## x scales::discard()      masks purrr::discard()
## x magrittr::extract()    masks tidyr::extract()
## x dplyr::filter()        masks stats::filter()
## x recipes::fixed()       masks stringr::fixed()
## x dplyr::lag()           masks stats::lag()
## x magrittr::set_names()  masks purrr::set_names()
## x yardstick::spec()      masks readr::spec()
## x recipes::step()        masks stats::step()

library(nnet)          # multinomial regression
library(kernlab)       # support vector machine

##
## Attaching package: 'kernlab'
##
## The following object is masked from 'package:dials':
##
##   buffer
##
## The following object is masked from 'package:scales':
##
##   alpha
##
## The following object is masked from 'package:purrr':
##
##   cross
##
## The following object is masked from 'package:ggplot2':
##
##   alpha

library(ranger)        # random forest

## Warning: package 'ranger' was built under R version 4.4.3

```

```
library(caret)           # confusion matrix

## Warning: package 'caret' was built under R version 4.4.3
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following objects are masked from 'package:yardstick':
##
##   precision, recall, sensitivity, specificity
##
## The following object is masked from 'package:purrr':
##
##   lift
```

```
library(GGally)         # ggpairs

## Warning: package 'GGally' was built under R version 4.4.3
## Registered S3 method overwritten by 'GGally':
##   method from
##   +.gg      ggplot2
```

```
library(kknn)           # KNN

## Warning: package 'kknn' was built under R version 4.4.3
##
## Attaching package: 'kknn'
##
## The following object is masked from 'package:caret':
##
##   contr.dummy
```

```
rm(list = ls())
```

Data loading

```
nba_data <- read_csv("current_nba_players.csv")

## New names:
## Rows: 526 Columns: 105
## -- Column specification
## ----- Delimiter: "," chr
## (53): name, nationality_1, nationality_2, team, jersey, position_1, posi... dbl
## (52): ...1, height_cm, weight_lbs, weight_kg, wingspan_cm, season_salary...
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
```

Data preprocessing

First of all, the data contains information about height, weight and wingspan in two metric systems so we will drop feet, lbs and will keep cm, kg. We can also drop hometown because it contains duplicat information of column nationality. Second nationality has too many missing values so we will drop the column.

```
nba_data <- nba_data %>%
  select(-height_feet, -weight_lbs, -wingspan_feet) %>%
  rename(height = height_cm, weight = weight_kg, wingspan = wingspan_cm) %>%
  select(-hometown, -nationality_2)
```

Now we will parse birthday into separate day, month and year - so we can check if maybe for some reason players born in some month are more likely to play some position.

```
nba_data <- nba_data %>%
  mutate(
    birthdate = lubridate::mdy(birthdate),
    birth_day = lubridate::day(birthdate),
    birth_month = lubridate::month(birthdate),
    birth_year = lubridate::year(birthdate)
  ) %>%
  relocate(birth_day, birth_month, birth_year, .after = birthdate) %>%
  select(-birthdate)
```

Since the data does not contain information about age we will compute the age based on birth date. The game from which the data came from was released September 6, 2024, this will be our anchor date to compute the age.

```
anchor_date <- lubridate::ymd("2024-09-06")
nba_data <- nba_data %>%
  mutate(
    age = lubridate::year(anchor_date) - birth_year -
      if_else(
        birth_month > lubridate::month(anchor_date) |
        (birth_month == lubridate::month(anchor_date) & birth_day > lubridate::day(anchor_date)),
        1, 0
      )
  ) %>%
  relocate(age, .after = birth_year)
```

Last columns from 64 to 103, are corresponding badges of some skills of the players, we will encode them to numerical values.

```
nba_data <- nba_data %>%
  mutate(across(64:103, ~ case_when(
    .x == "bronze" ~ 1,
    .x == "silver" ~ 2,
    .x == "gold" ~ 3,
    TRUE ~ 0
  )))
```

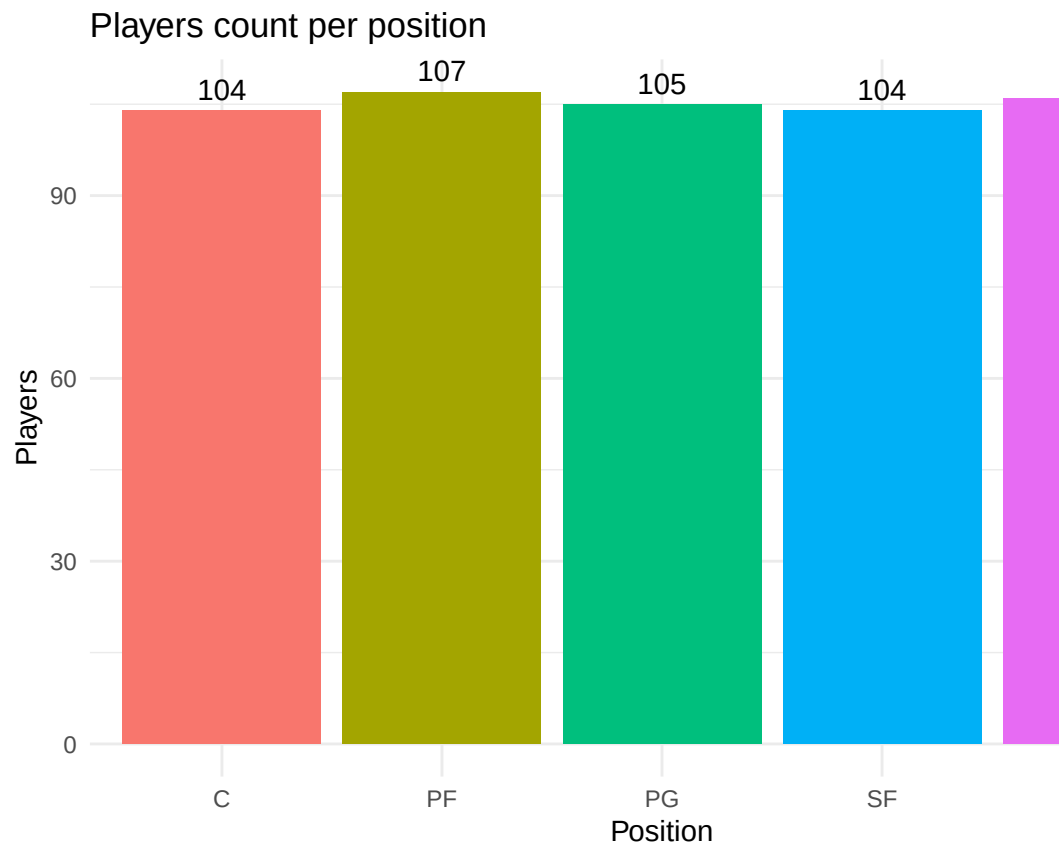
EDA

Our main focus will be classification per player position. We will drop secondary position because our primary goal is the main position.

```
nba_data <- nba_data %>% select(-position_2)

nba_data %>% count(position_1) %>%
  ggplot(aes(x=position_1, y=n, fill=position_1)) +
  geom_col() +
```

```
geom_text(aes(label = n), vjust = -0.5, size = 4) +
labs(
  title = "Players count per position",
  x = "Position",
  y = "Players"
) +
theme_minimal() +
theme(legend.position = "none")
```



Player position distribution

We can see that the player position classes are perfectly balanced. That means great start for classification problem.

```
# Boxplot for weight by position
weight_plot <- nba_data %>%
  ggplot(aes(x = position_1, y = weight, fill = position_1)) +
  geom_boxplot() +
  labs(
    title = "Weight distribution per position",
    x = "Position",
    y = "Weight"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

# Boxplot for height by position
```

```

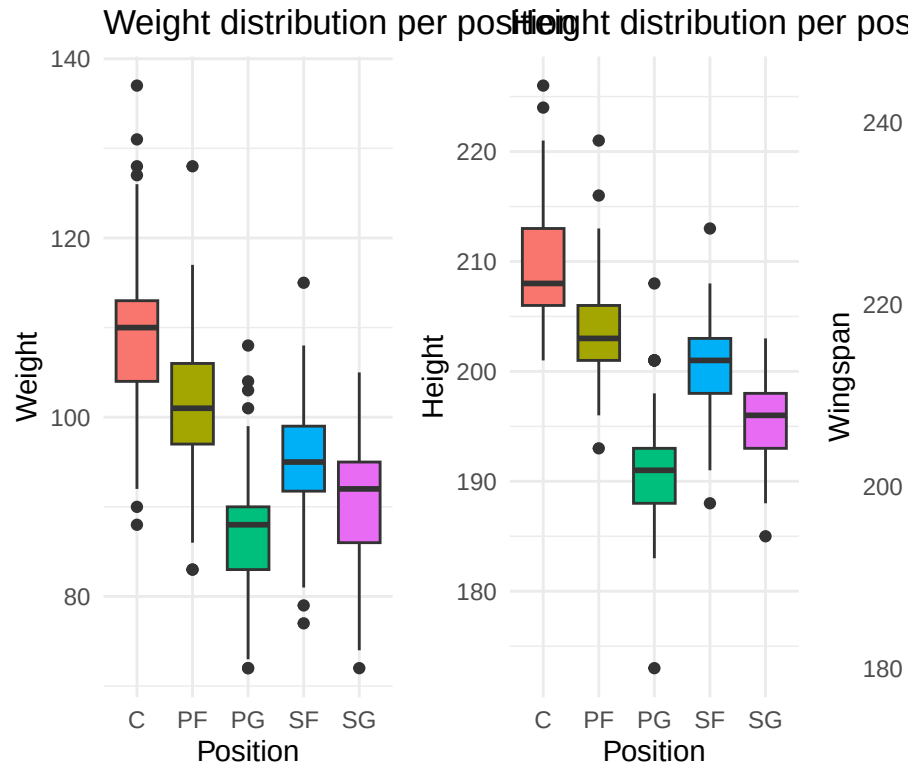
height_plot <- nba_data %>%
  ggplot(aes(x = position_1, y = height, fill = position_1)) +
  geom_boxplot() +
  labs(
    title = "Height distribution per position",
    x = "Position",
    y = "Height"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

# 'Boxplot for wingspan by position'
wingspan_plot <- nba_data %>%
  ggplot(aes(x = position_1, y = wingspan, fill = position_1)) +
  geom_boxplot() +
  labs(
    title = "Wingspan distribution per position",
    x = "Position",
    y = "Wingspan"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

(weight_plot | height_plot | wingspan_plot) +
  plot_layout(guides = "collect") +
  plot_annotation(title = "Player physical attributes per position")

```

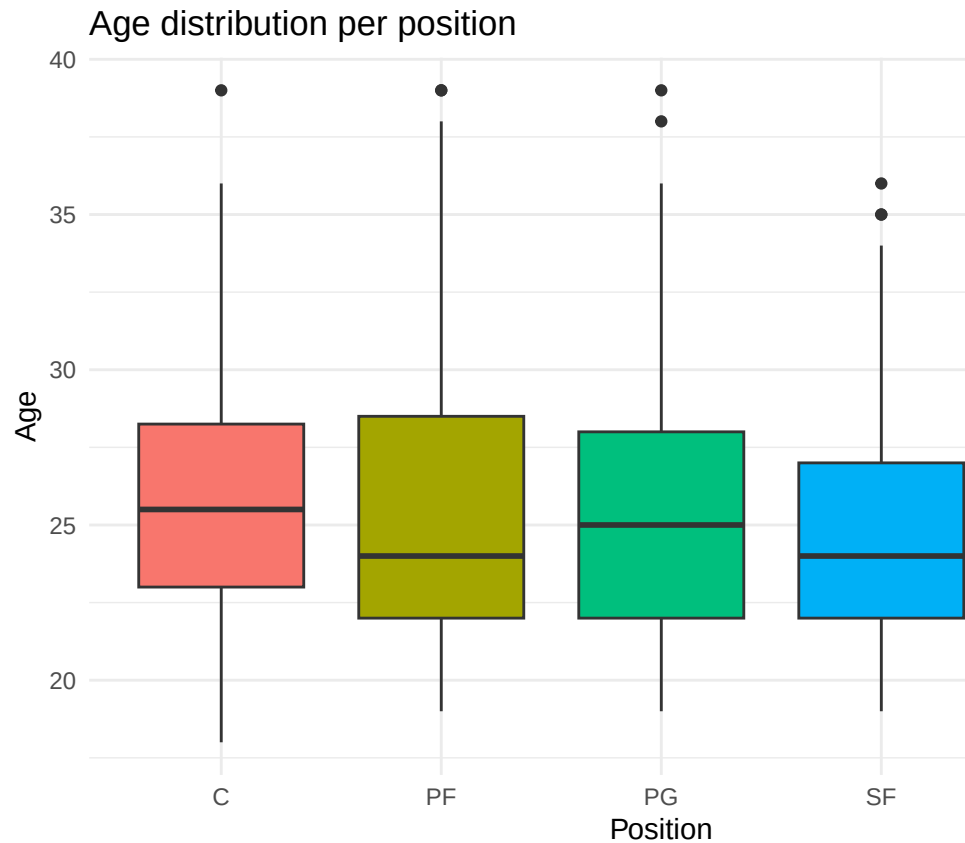
Player physical attributes per position



Player physical attributes per position

It looks like that PG - Point Guard players are by average smaller and lighter then the rest. It looks like that C - Center players are by average taller and heavier then the rest. Other positions hover between them.

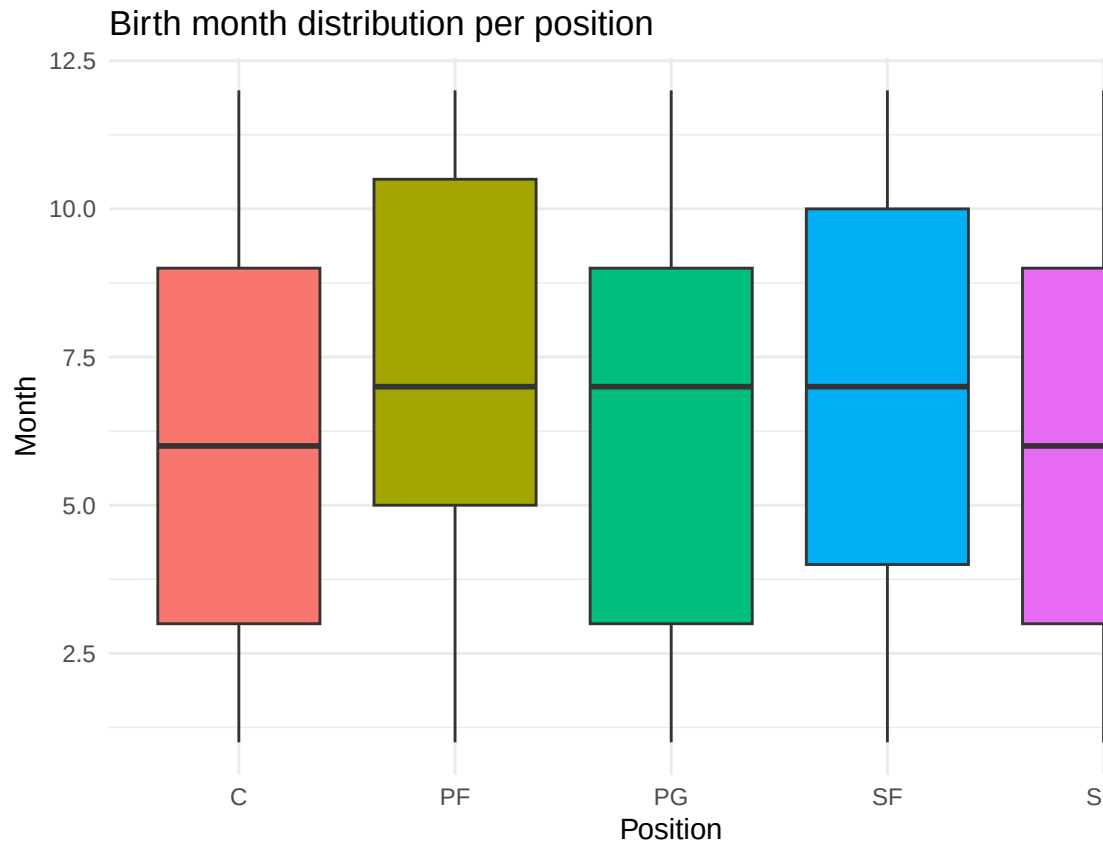
```
nba_data %>%
  ggplot(aes(x = position_1, y = age, fill = position_1)) +
  geom_boxplot() +
  labs(
    title = "Age distribution per position",
    x = "Position",
    y = "Age"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```



Age distribution between positions

Age doesn't seem to play any role in the player positions.

```
nba_data %>%  
  ggplot(aes(x = position_1, y = birth_month, fill = position_1)) +  
  geom_boxplot() +  
  labs(  
    title = "Birth month distribution per position",  
    x = "Position",  
    y = "Month"  
  ) +  
  theme_minimal() +  
  theme(legend.position = "none")
```

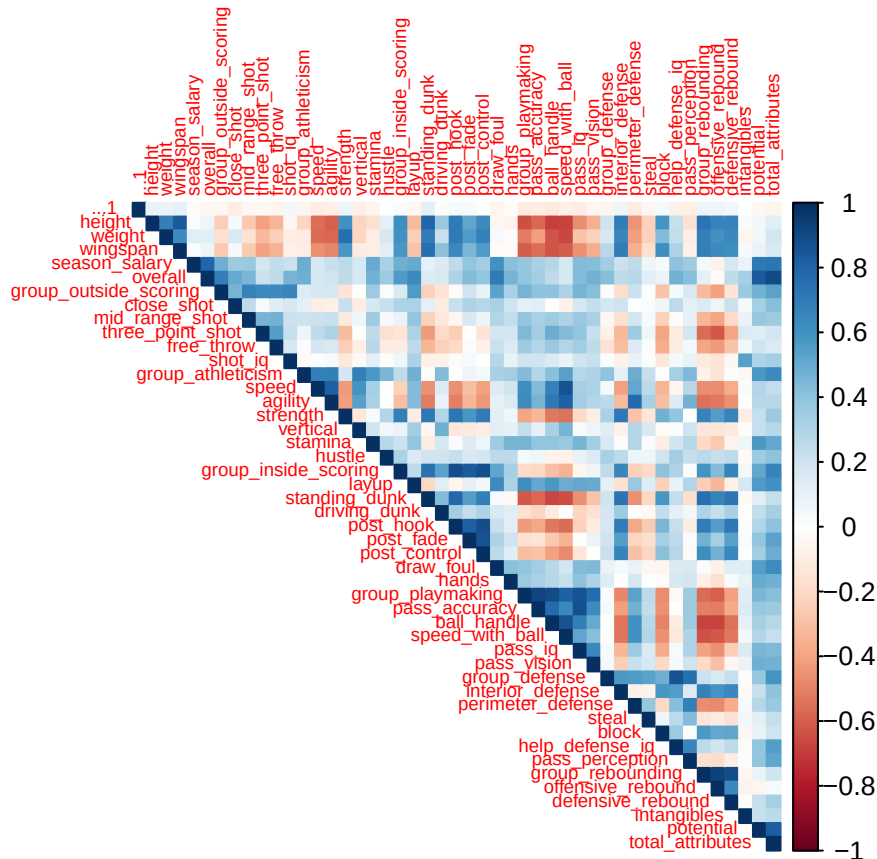


Birth month by position

There is no secret funny difference in position by player birth month.

Corplot visualization Visualizing the cor plot for better understanding of which attributes to select for feature selection.

```
nba_data_clean <- nba_data %>% # keep only attributes that make sense
  select(-contains("badge"), -contains("birth"), -contains("age"),
        -contains("years"), -contains("consistency"), -contains("durability"))
numeric_vars <- nba_data_clean %>% select(where(is.numeric)) # selecting only numeric
cor_matrix <- cor(numeric_vars, use = "complete.obs") # computing corplot
corrplot(cor_matrix, method = "color", type = "upper", tl.cex = 0.6, number.cex = 0.5)
```



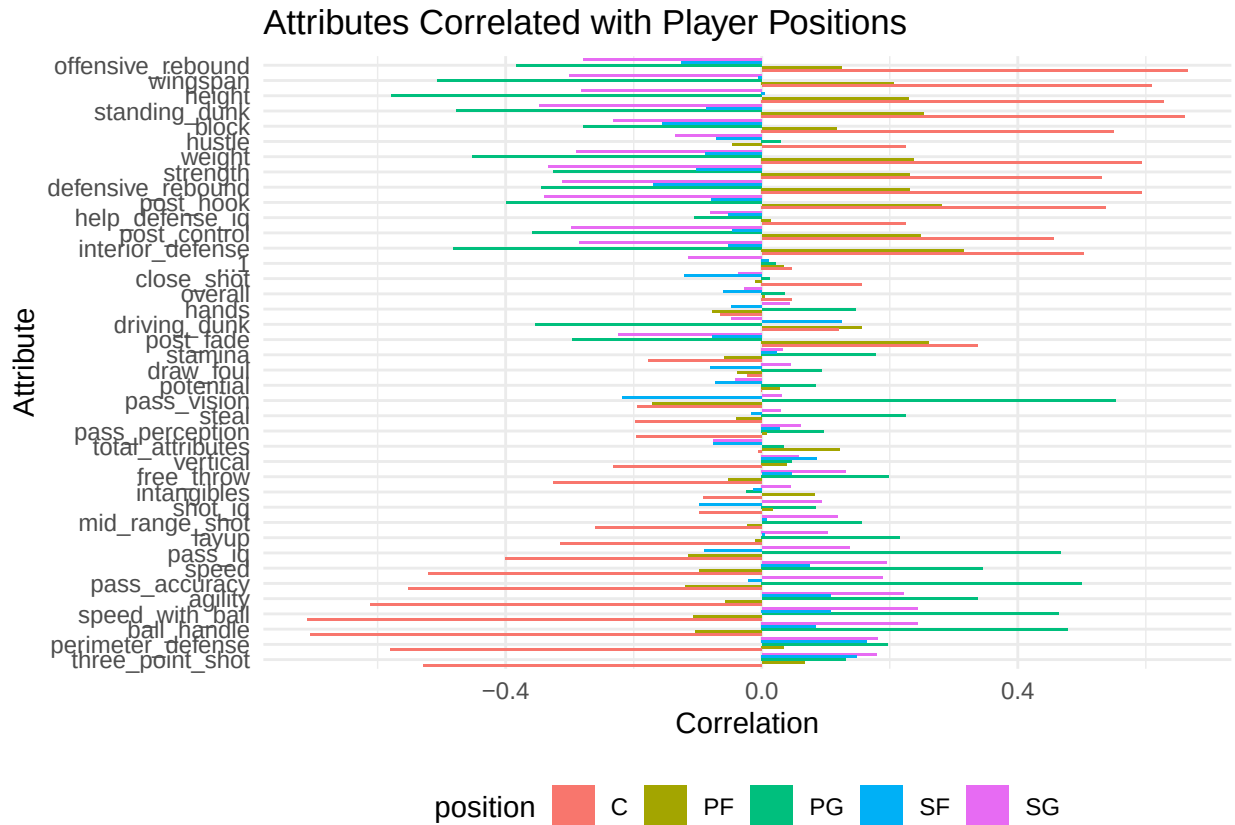
Visualizing correlating attributes in regards to position

```
nba_data_encoded <- nba_data %>%
  mutate(across(position_1, as.factor)) %>%
  mutate(PG = ifelse(position_1 == "PG", 1, 0),
         SG = ifelse(position_1 == "SG", 1, 0),
         SF = ifelse(position_1 == "SF", 1, 0),
         PF = ifelse(position_1 == "PF", 1, 0),
         C = ifelse(position_1 == "C", 1, 0))
nba_data_cleaned <- nba_data_encoded %>% # keep only attributes that make sense
  select(-contains("badge"), -contains("group"), -contains("birth"),
        -contains("age"), -contains("years"), -contains("consistency"),
        -contains("salary"), -contains("durability"))
corrnb_numeric_vars <- nba_data_cleaned %>% select(where(is.numeric))
cor_matrix <- cor(corrnb_numeric_vars, use = "complete.obs")
positions <- c("PG", "SG", "SF", "PF", "C") # extract correlations for each pos
cor_list <- lapply(positions, function(pos) {
  tibble(attribute = rownames(cor_matrix),
         correlation = cor_matrix[, pos],
         position = pos)
})
cor_long <- bind_rows(cor_list) %>% # combine into one long data frame
  filter(!(attribute %in% positions)) # exclude the one-hot columns themselves
ggplot(cor_long, aes(x = reorder(attribute, correlation), y = correlation, fill = position)) +
  geom_col(position = "dodge") +
  coord_flip() +
  labs(
```

```

title = "Attributes Correlated with Player Positions",
x = "Attribute",
y = "Correlation"
) +
theme_minimal() +
theme(legend.position = "bottom")

```



From this we can see that SFs and PFs are very average at everything, they have almost no significant correlations (makes sense, since they are jack of all trades types). Cs, SGs and PGs have nice strong correlations in regards to their responsibilities on the court (playmaking for PGs, shooting for SGs, shot blocking, interior offense & defense for Cs). It also looks like that PGs are opposite players to Cs.

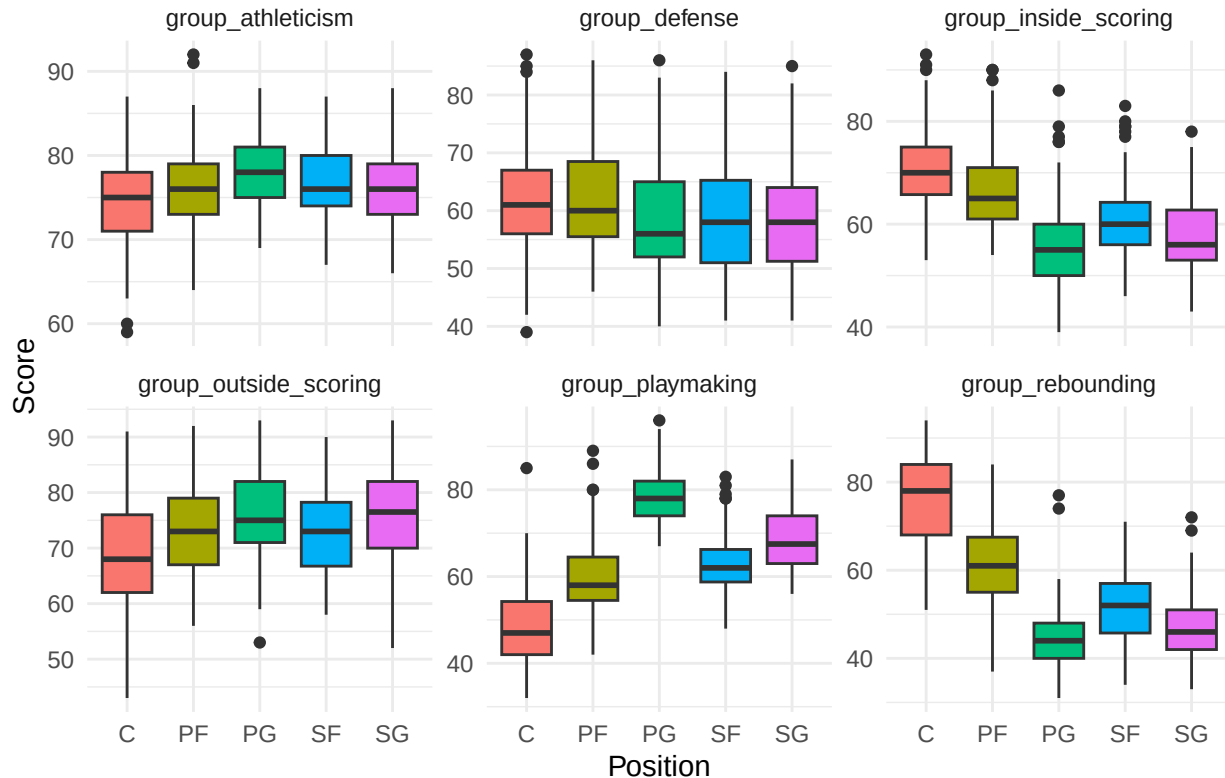
Skill groups visualization Visualizing which position is better at what skill group to better understand the differences between the positions.

```

nba_data %>%
  pivot_longer(cols = starts_with("group_"), names_to = "skill_group", values_to = "value") %>%
  ggplot(aes(x = position_1, y = value, fill = position_1)) +
  geom_boxplot() +
  facet_wrap(~ skill_group, scales = "free_y") +
  labs(title = "Skill Group Distribution per Position", x = "Position", y = "Score") +
  theme_minimal() +
  theme(legend.position = "none")

```

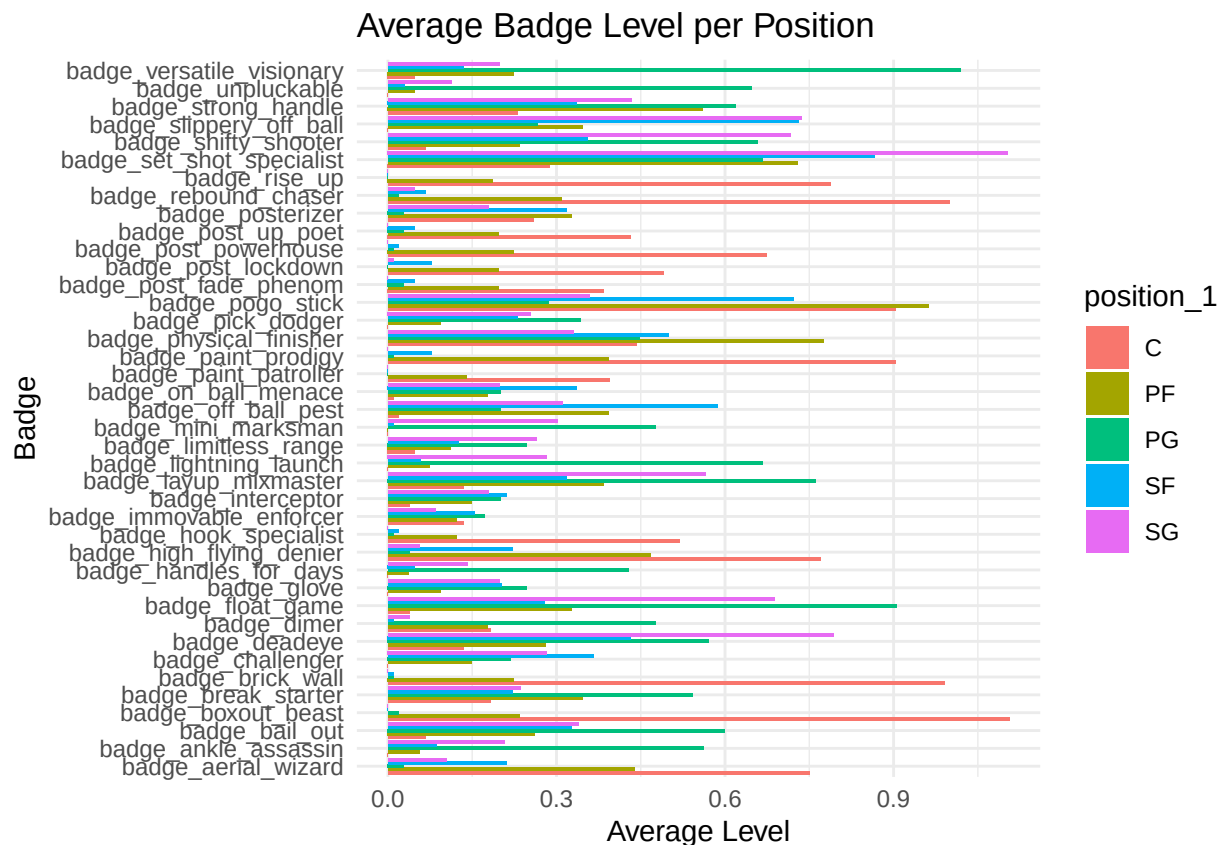
Skill Group Distribution per Position



From the data it looks like point guards are the playmakers, centers are the rebounders, outside shooting is dominated by shooting & point guards, inside scoring by centers and power forwards and defense is evenly distributed. Small forwards are again average at everything.

Badge distribution among positions Visualizing what badges are most common for which position.

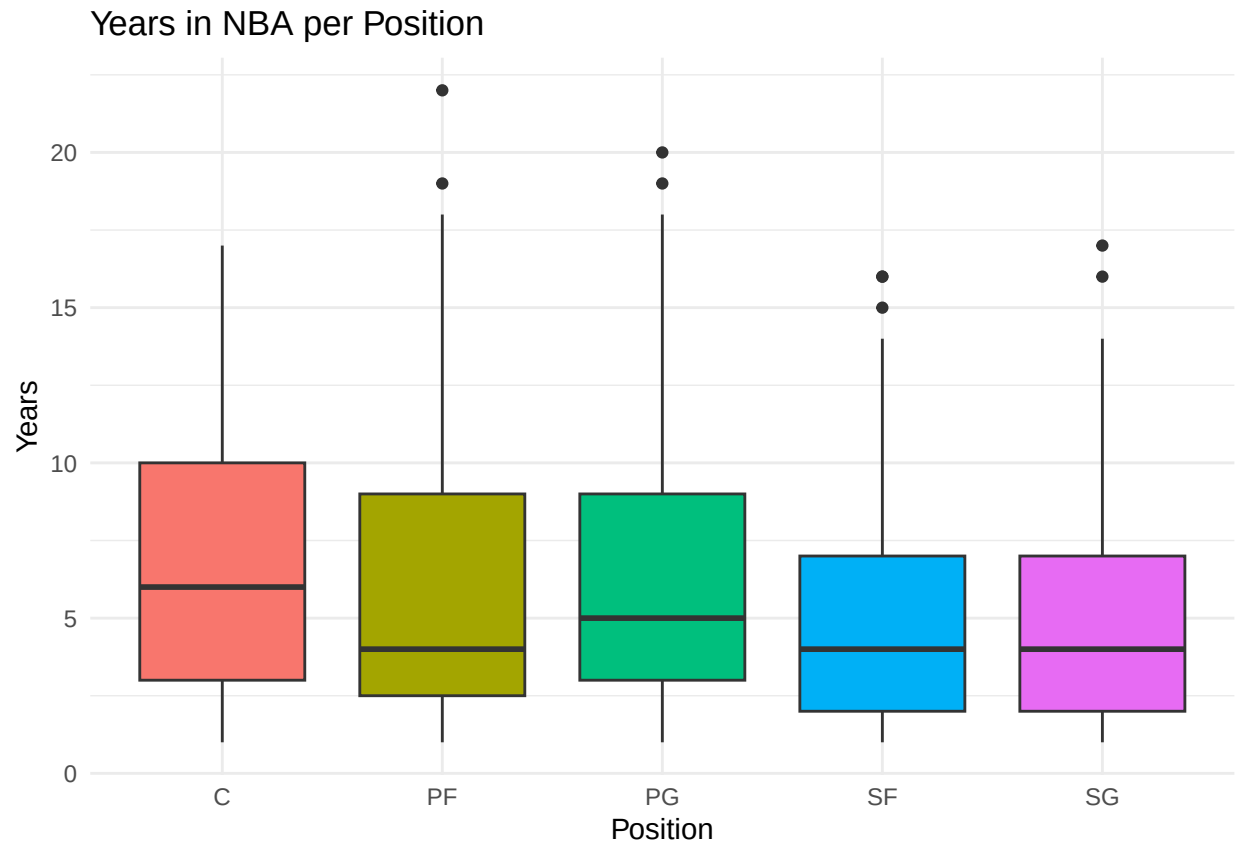
```
badge_cols <- nba_data %>% select(starts_with("badge_")) %>% names()
badge_summary <- nba_data %>%
  select(position_1, all_of(badge_cols)) %>%
  pivot_longer(cols = all_of(badge_cols), names_to = "badge", values_to = "level") %>%
  group_by(position_1, badge) %>%
  summarise(avg_level = mean(level, na.rm=TRUE), .groups = 'drop')
badge_summary %>%
  ggplot(aes(x = avg_level, y = badge, fill = position_1)) +
  geom_col(position = "dodge") +
  labs(title = "Average Badge Level per Position", x = "Average Level", y = "Badge") +
  theme_minimal()
```



Rookie & veteran distribution Visualizing if some positions are usually played by rookies or veterans.

```
nba_data %>%
  ggplot(aes(x = position_1, y = years_in_the_nba, fill = position_1)) +
  geom_boxplot() +
  labs(title = "Years in NBA per Position", x = "Position", y = "Years") +
  theme_minimal() +
  theme(legend.position = "none")
```

```
## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_boxplot()`).
```



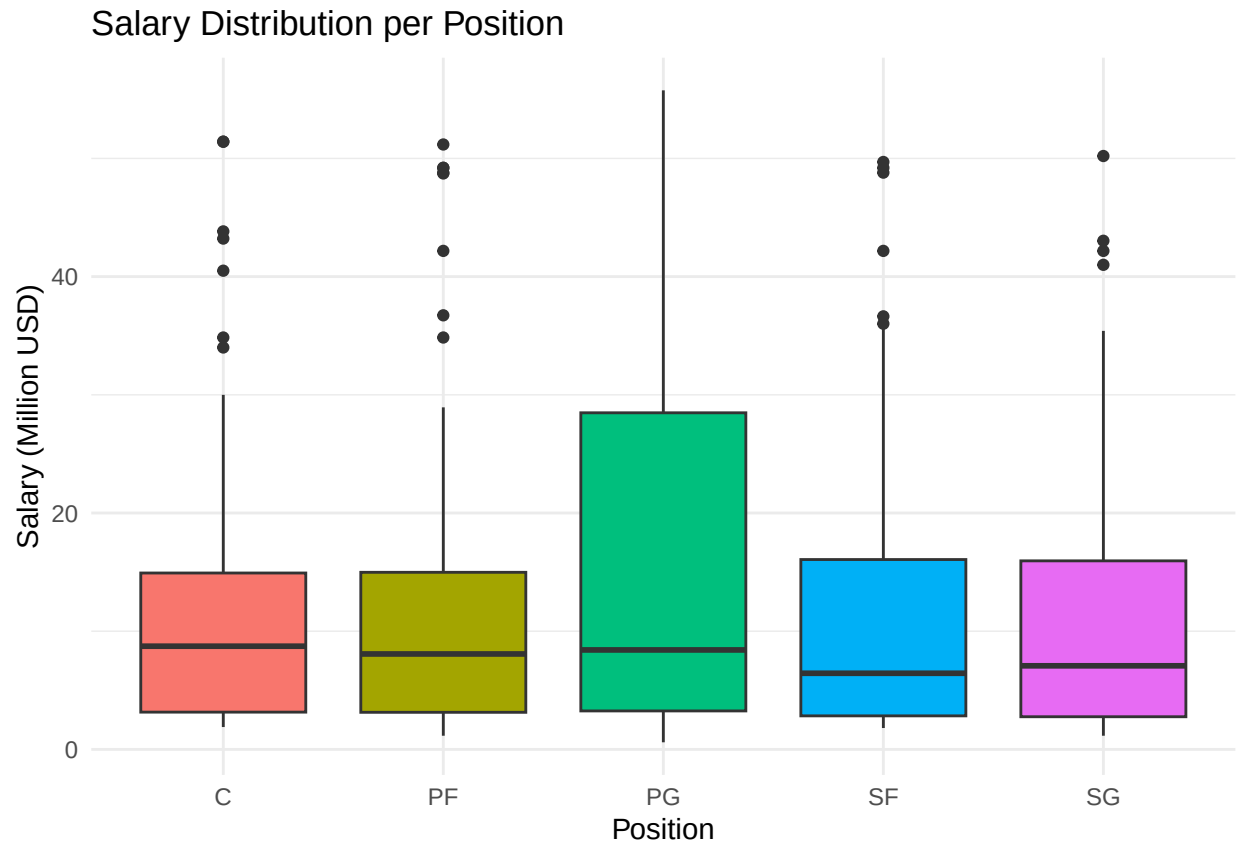
Does not seem to be anything of value from this.

Salary distribution per position Visualizing if salaries differ per positions.

```
nba_data %>%
  ggplot(aes(x = position_1, y = season_salary/1e6, fill = position_1)) +
  geom_boxplot() +
  labs(title = "Salary Distribution per Position", x = "Position", y = "Salary (Million USD)") +
  theme_minimal() +
  theme(legend.position = "none")
```

```
## Warning: Removed 127 rows containing non-finite outside the scale range
```

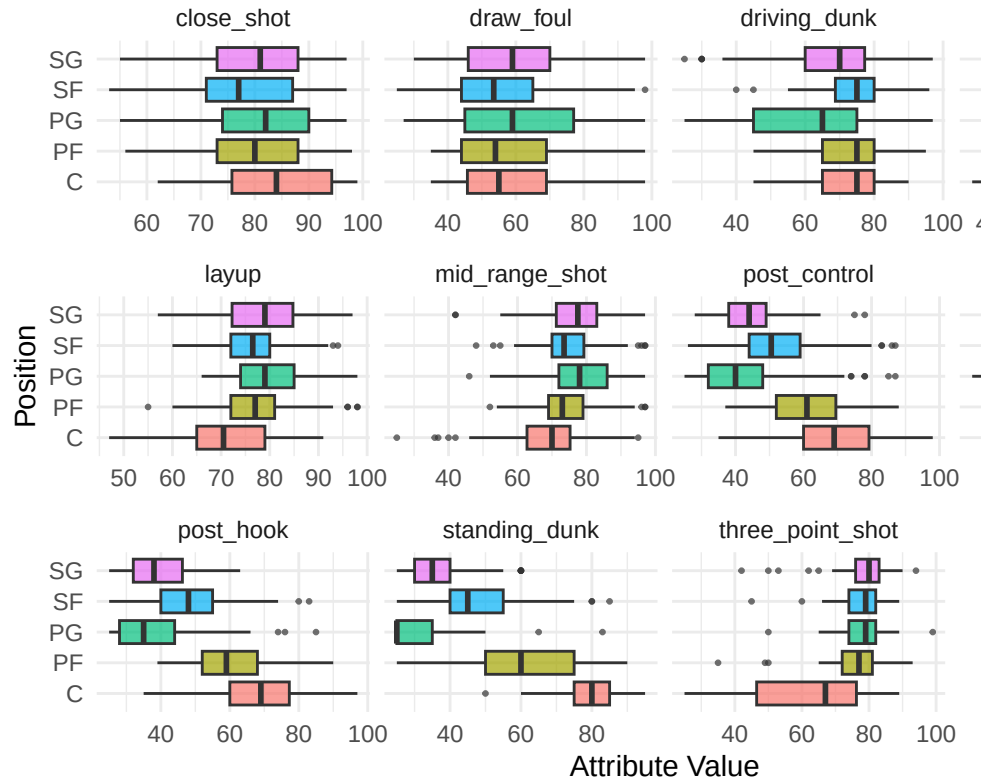
```
## (`stat_boxplot()`).
```



Other positions seem to be on the same level except for point guards, which are paid the most by far (10 million difference in upper limit), which makes sense, since they have the responsibility of initiating the whole offense for the team.

```
nba_data %>%
  select(position_1, close_shot, mid_range_shot, three_point_shot, free_throw,
         draw_foul, layup, driving_dunk, standing_dunk, post_fade, post_hook,
         post_control) %>%
  pivot_longer(-position_1, names_to = "offensive_skill", values_to = "value") %>%
  ggplot(aes(x = value, y = position_1, fill = position_1)) +
  geom_boxplot(alpha = 0.7, outlier.size = 0.5) +
  facet_wrap(~ offensive_skill, scales = "free_x") +
  labs(title = "Offensive Attributes by Position", x = "Attribute Value", y = "Position") +
  theme_minimal() +
  theme(legend.position = "none")
```

Offensive Attributes by Position

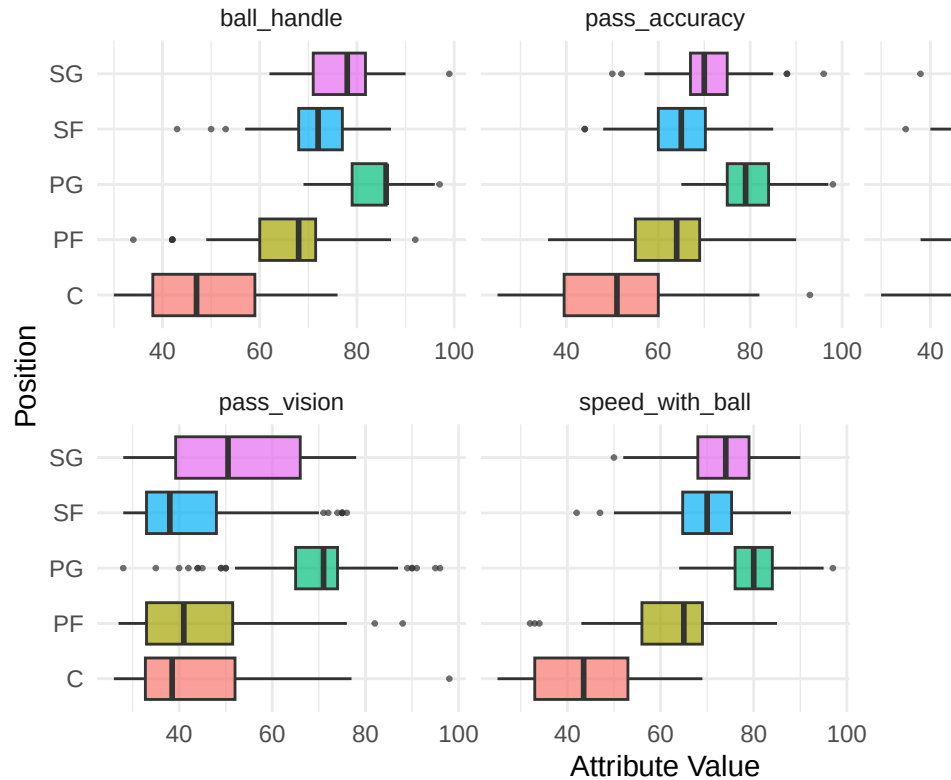


Offensive attributes per positions

From the data it looks like centers are the best at close shot, bad at freethrows and unreliable shooters. On the other hand, point & shooting guards are great at shooting, and forwards are jack of all trades. Attribute post hook and post fade seems to be best to divide the players into two groups: [PF, C] and [SG, SF, PG].

```
nba_data %>%
  select(position_1, pass_accuracy, ball_handle, speed_with_ball, pass_iq, pass_vision) %>%
  pivot_longer(-position_1, names_to = "playmaking_skill", values_to = "value") %>%
  ggplot(aes(x = value, y = position_1, fill = position_1)) +
  geom_boxplot(alpha = 0.7, outlier.size = 0.5) +
  facet_wrap(~ playmaking_skill, scales = "free_x") +
  labs(title = "Playmaking Attributes by Position", x = "Attribute Value", y = "Position") +
  theme_minimal() +
  theme(legend.position = "none")
```

Playmaking Attributes by Position

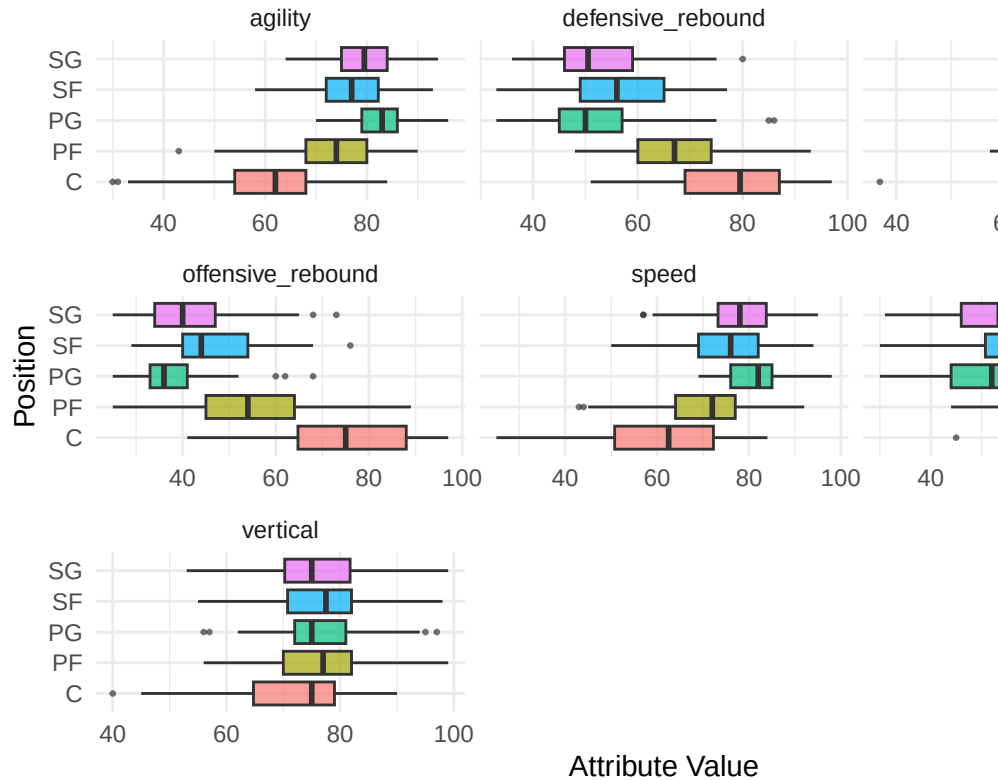


Playmaking attributes per positions

From the data we can assume that point guards are the best at handling the ball, fastest and best at overall passing. Centers are bad at play making and slow by the looks of it. Forwards are again bad at play making, shooting guards are decent. The best attribute to divide the players seems like to be pass and handle.

```
nba_data %>%
  select(position_1, speed, agility, strength, vertical, hands,
         offensive_rebound, defensive_rebound) %>%
  pivot_longer(-position_1, names_to = "athletic_skill", values_to = "value") %>%
  ggplot(aes(x = value, y = position_1, fill = position_1)) +
  geom_boxplot(alpha = 0.7, outlier.size = 0.5) +
  facet_wrap(~ athletic_skill, scales = "free_x") +
  labs(title = "Athletic Attributes by Position", x = "Attribute Value", y = "Position") +
  theme_minimal() +
  theme(legend.position = "none")
```

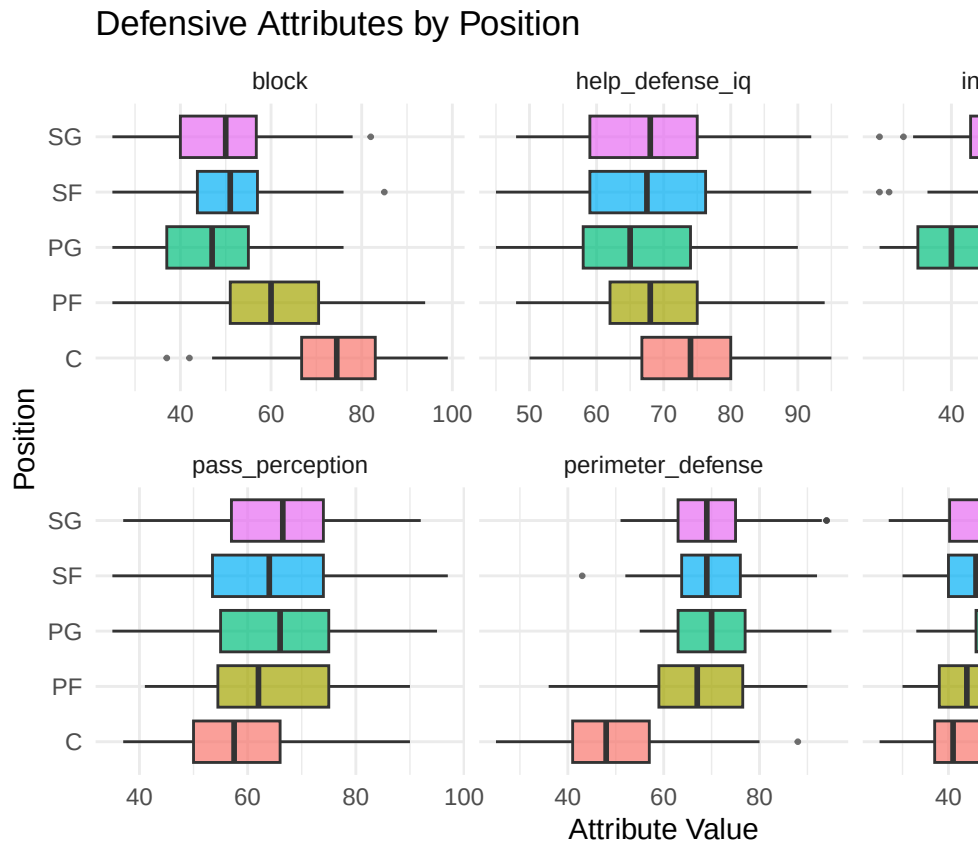
Athletic Attributes by Position



Athletic attributes per positions

From the data we can assume that PGs & SGs are the fastest, but the weakest (which makes sense for a small stature player). On the other hand centers are the strongest, but least athletic. Forwards are again average at all categories. We can make an assumption from the data that if player is strong then in general is slow. If player is fast and athletic then in general is not very strong. The best attribute to divide the players seems to be the strength attribute.

```
nba_data %>%
  select(position_1, interior_defense, perimeter_defense, steal, block, help_defense_iq, pass_perception) %>%
  pivot_longer(-position_1, names_to = "defensive_skill", values_to = "value") %>%
  ggplot(aes(x = value, y = position_1, fill = position_1)) +
  geom_boxplot(alpha = 0.7, outlier.size = 0.5) +
  facet_wrap(~ defensive_skill, scales = "free_x") +
  labs(title = "Defensive Attributes by Position", x = "Attribute Value", y = "Position") +
  theme_minimal() +
  theme(legend.position = "none")
```



Defensive attributes per positions

Cs and PFs look defensively oriented in the interior defense and blocks (which makes sense, since they are the tallest) and PGs and SGs are better perimeter defenders (makes sense, they are quicker, can steal the ball easier), SFs are average at all categories, jack of all trades type. These defensive attributes seems to highly correlate with how tall the player is. If player is tall (PF or C), they are good at blocking shots and interior defense. If players are short, they generally are good at perimeter defense and stealing the ball (because of their quickness). We can divide the players by height attribute.

Picked Predictors

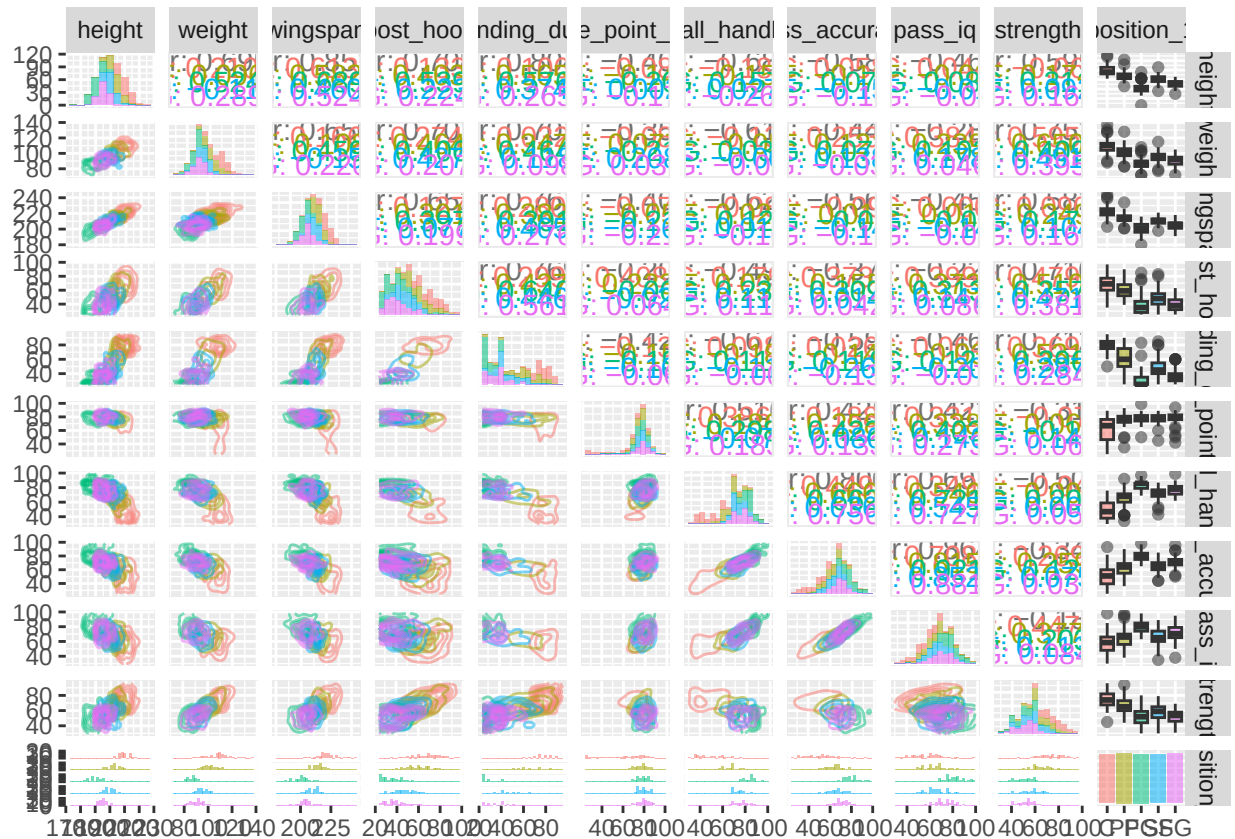
From the EDA above it seems like the best predictors to choose are: height, weight, wingspan, post_hook, standing_dunk, three_point_shot, ball_handle, pass_accuracy, pass_iq, strength.

We will visualize the picked data to better see which to choose further.

```
picked_data <- nba_data %>%
  select(height, weight, wingspan,
         post_hook, standing_dunk, three_point_shot,
         ball_handle, pass_accuracy, pass_iq,
         strength,
         position_1)

ggpairs(
  picked_data,
  aes(color = position_1, alpha = 0.5),
  lower = list(continuous = wrap("density", alpha = 0.5)),
  diag = list(continuous = wrap("barDiag", binwidth = 5))
)
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



Strong correlation between height and wingspan, we will drop one. Second strong correlation between height and standing dunk, we will drop height. Strong correlation between pass_accuracy and pass IQ, we will drop one.

```
picked_data <- picked_data %>%
  select(-height, -pass_iq)
```

Training

```
set.seed(123)
# prepare data with only selected columns decided from EDA
nba_model_data <- picked_data %>%
  mutate(position_1 = as.factor(position_1))
# split into train and test
split <- initial_split(nba_model_data, prop=0.8, strata = position_1)
```

```

train_data <- training(split)
test_data <- testing(split)
# formula for models (get columns except position_1)
predictor_columns <- setdiff(names(nba_model_data), "position_1")
# build the formula
model_formula <- as.formula(paste("position_1 ~", paste(predictor_columns, collapse = " + ")))

```

Data splitting We think that for our task is really important interpretability because we want to know and tell how and which attributes are important for each role. Because of this our main goal is to choose models which are high in interpretability.

```

lr_model <- multinom_reg(mode = "classification", engine = "nnet",
                        penalty = tune())

lr_workflow <- workflow() %>%
  add_model(lr_model) %>%
  add_formula(model_formula)

# Grid search with 5 fold cross validation
lr_resamples <- vfold_cv(train_data, v = 5)

lr_grid <- grid_regular(
  penalty(range = c(-6, 0)),
  levels = 50)

lr_tune <- tune_grid(
  lr_workflow,
  resamples = lr_resamples,
  grid = lr_grid,
  metrics = metric_set(accuracy)
)

best_lr <- select_best(lr_tune, metric = "accuracy")

# Final fit
final_lr_workflow <- finalize_workflow(lr_workflow, best_lr)

lr_fit <- fit(final_lr_workflow, data = train_data)

lr_train_preds <- predict(lr_fit, train_data) %>% bind_cols(train_data)
lr_test_preds <- predict(lr_fit, test_data) %>% bind_cols(test_data)

lr_train_metrics <- metrics(lr_train_preds, truth = position_1, estimate = .pred_class) %>%
  mutate(model = "Multinomial Regression", dataset = "train")
lr_test_metrics <- metrics(lr_test_preds, truth = position_1, estimate = .pred_class) %>%
  mutate(model = "Multinomial Regression", dataset = "test")

```

Logistic regression - training

```

svm_model <- svm_linear(mode = "classification", engine = "kernlab",
                      cost = tune())

```

```

svm_workflow <- workflow() %>%
  add_model(svm_model) %>%
  add_formula(model_formula)

# Grid search with 5 fold cross validation
svm_resamples <- vfold_cv(train_data, v = 5)

svm_grid <- grid_regular(
  cost(range = c(-10, 5)),
  levels = 30
)

svm_tune <- tune_grid(
  svm_workflow,
  resamples = svm_resamples,
  grid = svm_grid,
  metrics = metric_set(accuracy)
)

best_svm <- select_best(svm_tune, metric = "accuracy")

# Final fit
final_svm_workflow <- finalize_workflow(svm_workflow, best_svm)
final_svm_fit <- fit(final_svm_workflow, data = train_data)

```

Linear SVM - training

Setting default kernel parameters

```

svm_train_preds <- augment(final_svm_fit, new_data = train_data)
svm_test_preds <- augment(final_svm_fit, new_data = test_data)

svm_train_metrics <- metrics(svm_train_preds, truth = position_1, estimate = .pred_class) %>%
  mutate(model = "SVM-Linear", dataset = "train")
svm_test_metrics <- metrics(svm_test_preds, truth = position_1, estimate = .pred_class) %>%
  mutate(model = "SVM-Linear", dataset = "test")

```

```

svmp_model <- svm_poly(mode = "classification", engine = "kernlab",
  cost = tune(), degree = tune(), scale_factor = tune())

svmp_workflow <- workflow() %>%
  add_model(svmp_model) %>%
  add_formula(model_formula)

# Grid search with 5 fold cross validation
svmp_resamples <- vfold_cv(train_data, v = 5)

svmp_grid <- grid_regular(
  cost(range = c(-3, 3)),
  degree(range = c(2, 4)),
  scale_factor(range = c(-3, 0)),
  levels = 3
)

```

```

svmp_tune <- tune_grid(
  svmp_workflow,
  resamples = svmp_resamples,
  grid = svmp_grid,
  metrics = metric_set(accuracy)
)

best_svmp <- select_best(svmp_tune, metric = "accuracy")

# Final fit
final_svmp_workflow <- finalize_workflow(svmp_workflow, best_svmp)
final_svmp_fit <- fit(final_svmp_workflow, data = train_data)

svmp_train_preds <- augment(final_svmp_fit, new_data = train_data)
svmp_test_preds <- augment(final_svmp_fit, new_data = test_data)

svmp_train_metrics <- metrics(svmp_train_preds, truth = position_1, estimate = .pred_class) %>%
  mutate(model = "SVM-Polynomial", dataset = "train")
svmp_test_metrics <- metrics(svmp_test_preds, truth = position_1, estimate = .pred_class) %>%
  mutate(model = "SVM-Polynomial", dataset = "test")

```

Polynomial SVM - training

```

svmrbf_model <- svm_rbf(mode = "classification", engine = "kernlab",
  cost = tune(), rbf_sigma = tune())

svmrbf_workflow <- workflow() %>%
  add_model(svmrbf_model) %>%
  add_formula(model_formula)

# Grid search with 5 fold cross validation
svmrbf_resamples <- vfold_cv(train_data, v = 5)

svmrbf_grid <- grid_regular(
  cost(range = c(-10, 5)),
  rbf_sigma(range = c(-5, 0)),
  levels = 5
)

svmrbf_tune <- tune_grid(
  svmrbf_workflow,
  resamples = svmrbf_resamples,
  grid = svmrbf_grid,
  metrics = metric_set(accuracy)
)

best_svmrbf <- select_best(svmrbf_tune, metric = "accuracy")

# Final fit
final_svmrbf_workflow <- finalize_workflow(svmrbf_workflow, best_svmrbf)
final_svmrbf_fit <- fit(final_svmrbf_workflow, data = train_data)

```

```

svmrbf_train_preds <- augment(final_svmrbf_fit, new_data = train_data)
svmrbf_test_preds  <- augment(final_svmrbf_fit, new_data = test_data)

svmrbf_train_metrics <- metrics(svmrbf_train_preds, truth = position_1, estimate = .pred_class) %>%
  mutate(model = "SVM-RBF", dataset = "train")
svmrbf_test_metrics  <- metrics(svmrbf_test_preds, truth = position_1, estimate = .pred_class) %>%
  mutate(model = "SVM-RBF", dataset = "test")

```

RBF SVM - training

```

tree_model <- decision_tree( mode = "classification", engine = "rpart",
                             cost_complexity = tune(),
                             tree_depth = tune(),
                             min_n = tune()
                           )

tree_workflow <- workflow() %>%
  add_model(tree_model) %>%
  add_formula(model_formula)

# Grid search with 5 fold cross validation
tree_resamples <- vfold_cv(train_data, v = 5)

tree_grid <- grid_regular(
  cost_complexity(range = c(-4, -1)),
  tree_depth(range = c(1, 10)),
  min_n(range = c(1, 10)),
  levels = 5
)

tree_tune <- tune_grid(
  tree_workflow,
  resamples = tree_resamples,
  grid = tree_grid,
  metrics = metric_set(accuracy)
)

best_tree <- select_best(tree_tune, metric = "accuracy")

# Final fit
final_tree_workflow <- finalize_workflow(tree_workflow, best_tree)
final_tree_fit <- fit(final_tree_workflow, data = train_data)

tree_train_preds <- augment(final_tree_fit, new_data = train_data)
tree_test_preds  <- augment(final_tree_fit, new_data = test_data)

tree_train_metrics <- metrics(tree_train_preds, truth = position_1, estimate = .pred_class) %>%
  mutate(model = "Decision Tree", dataset = "train")
tree_test_metrics  <- metrics(tree_test_preds, truth = position_1, estimate = .pred_class) %>%
  mutate(model = "Decision Tree", dataset = "test")

```

Decision Tree - training

```
bind_rows(
  lr_train_metrics, lr_test_metrics,
  svm_train_metrics, svm_test_metrics,
  svmp_train_metrics, svmp_test_metrics,
  svmrbf_train_metrics, svmrbf_test_metrics,
  tree_train_metrics, tree_test_metrics
)
```

View training & testing metrics of trained models

```
## # A tibble: 20 x 5
##   .metric .estimator .estimate model          dataset
##   <chr>   <chr>      <dbl> <chr>      <chr>
## 1 accuracy multiclass 0.697 Multinomial Regression train
## 2 kap      multiclass 0.621 Multinomial Regression train
## 3 accuracy multiclass 0.692 Multinomial Regression test
## 4 kap      multiclass 0.614 Multinomial Regression test
## 5 accuracy multiclass 0.721 SVM-Linear          train
## 6 kap      multiclass 0.651 SVM-Linear          train
## 7 accuracy multiclass 0.701 SVM-Linear          test
## 8 kap      multiclass 0.626 SVM-Linear          test
## 9 accuracy multiclass 0.792 SVM-Polynomial       train
## 10 kap     multiclass 0.740 SVM-Polynomial       train
## 11 accuracy multiclass 0.626 SVM-Polynomial       test
## 12 kap     multiclass 0.532 SVM-Polynomial       test
## 13 accuracy multiclass 0.821 SVM-RBF            train
## 14 kap     multiclass 0.776 SVM-RBF            train
## 15 accuracy multiclass 0.607 SVM-RBF            test
## 16 kap     multiclass 0.509 SVM-RBF            test
## 17 accuracy multiclass 0.821 Decision Tree      train
## 18 kap     multiclass 0.776 Decision Tree      train
## 19 accuracy multiclass 0.551 Decision Tree      test
## 20 kap     multiclass 0.439 Decision Tree      test
```

From the result we can see that decision tree is heavily overfitted. SVM-Polynomial & SVM-RBF seem to be better than decision tree but are still overfitted. SVM-Linear seems to be very good, but looks slightly overfitted. The best model for this task seems to be Multinomial Logistic Regression where there is no sign of overfit and also has the best test results.

Evaluation

```
lr_test_predictions <- predict(lr_fit, test_data) %>%
  bind_cols(test_data)

confusionMatrix(lr_test_predictions$.pred_class, lr_test_predictions$position_1)

## Confusion Matrix and Statistics
##
##              Reference
## Prediction  C  PF  PG  SF  SG
##          C  15   4   0   0   0
##          PF   6  15   0   3   0
##          PG   0   0  17   1   4
##          SF   0   0   0  13   4
```

```
##          SG  0  3  4  4 14
##
## Overall Statistics
##
##          Accuracy : 0.6916
##          95% CI : (0.595, 0.7773)
##      No Information Rate : 0.2056
##      P-Value [Acc > NIR] : < 2.2e-16
##
##          Kappa : 0.6142
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##          Class: C Class: PF Class: PG Class: SF Class: SG
## Sensitivity      0.7143    0.6818    0.8095    0.6190    0.6364
## Specificity      0.9535    0.8941    0.9419    0.9535    0.8706
## Pos Pred Value   0.7895    0.6250    0.7727    0.7647    0.5600
## Neg Pred Value   0.9318    0.9157    0.9529    0.9111    0.9024
## Prevalence       0.1963    0.2056    0.1963    0.1963    0.2056
## Detection Rate   0.1402    0.1402    0.1589    0.1215    0.1308
## Detection Prevalence 0.1776    0.2243    0.2056    0.1589    0.2336
## Balanced Accuracy 0.8339    0.7880    0.8757    0.7863    0.7535
```

We can see that the accuracy on first sight does not seem to be that great but compared to the no information rate (increase of 48%) it looks really great. Also we can see that class C, PG are the easiest for the model to predict. Classes PF, SF and SG are making the most problems for the model.

ROC analysis Building of the best model with probabilities:

```
lr_model <- multinom_reg(mode = "classification", engine = "nnet",
                          penalty = best_lr$penalty)

lr_workflow <- workflow() %>%
  add_model(lr_model) %>%
  add_formula(model_formula)

lr_prob_fit <- fit(lr_workflow, data = train_data)
lr_train_probs <- predict(lr_prob_fit, train_data, type = "prob")
lr_test_probs <- predict(lr_prob_fit, test_data, type = "prob")
```

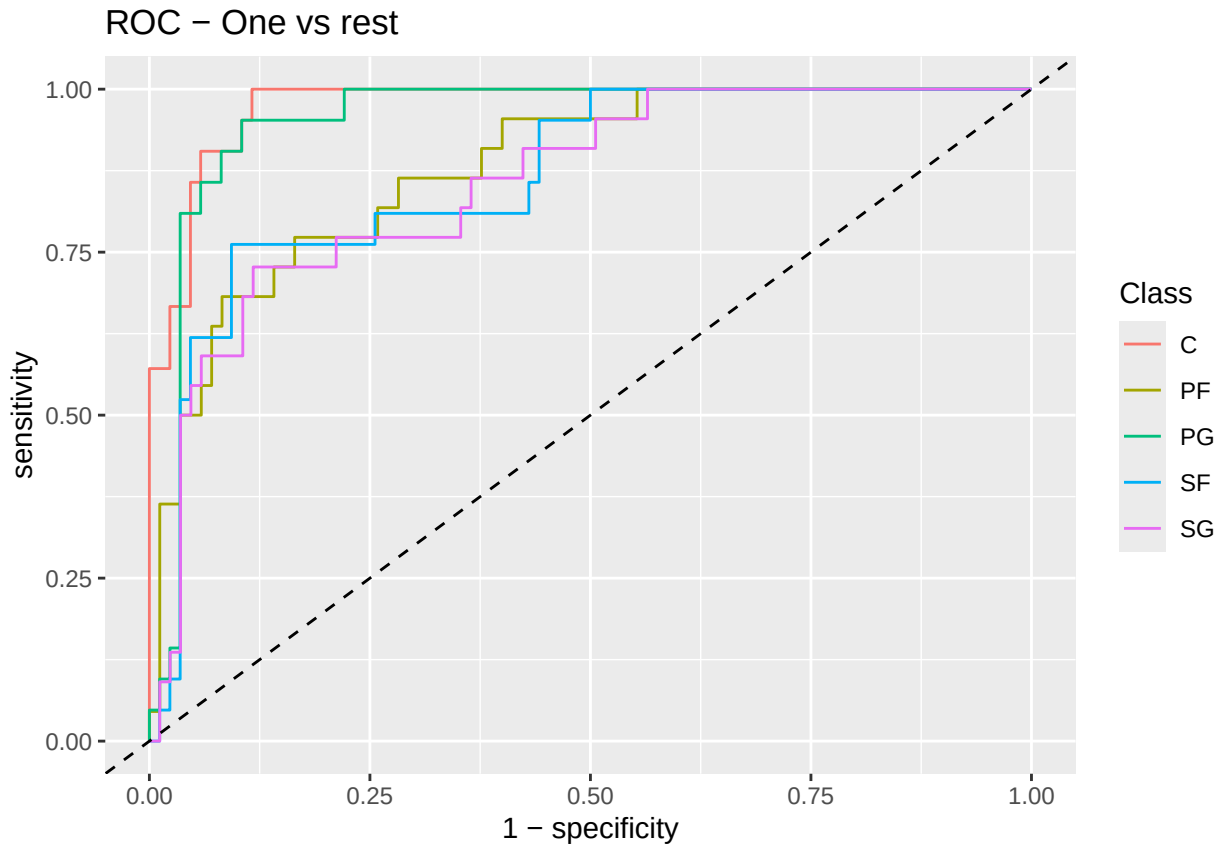
Computing ROC:

```
lr_test_probs <- lr_test_probs %>%
  bind_cols(truth = test_data$position_1)

test_roc <- roc_curve(lr_test_probs, truth = truth,
                      .pred_C, .pred_PF, .pred_PG, .pred_SF, .pred_SG)

test_roc %>%
  ggplot(aes(x = 1 - specificity, y = sensitivity, color = .level)) +
  geom_path() +
  geom_abline(lty = 2) +
  labs(title = "ROC - One vs rest",
```

```
color = "Class")
```



Computing Youden index per class:

```
lr_train_probs <- lr_train_probs %>%
  bind_cols(truth = train_data$position_1)

train_roc <- roc_curve(lr_train_probs, truth = truth,
  .pred_C, .pred_PF, .pred_PG, .pred_SF, .pred_SG)

best_thresholds <- train_roc %>%
  mutate(youden_index = sensitivity + specificity - 1) %>%
  group_by(.level) %>%
  slice_max(order_by = youden_index, n = 1, with_ties = FALSE) %>%
  select(.level, .threshold)
```

Applying the thresholds:

```
threshold_dict <- best_thresholds %>% deframe()

final_predictions <- lr_test_probs %>%
  mutate(
    C = if_else(.pred_C >= threshold_dict["C"], .pred_C, 0),
    PF = if_else(.pred_PF >= threshold_dict["PF"], .pred_PF, 0),
    PG = if_else(.pred_PG >= threshold_dict["PG"], .pred_PG, 0),
    SF = if_else(.pred_SF >= threshold_dict["SF"], .pred_SF, 0),
    SG = if_else(.pred_SG >= threshold_dict["SG"], .pred_SG, 0)
```

```

) %>%
select(C, PF, PG, SF, SG) %>%
rowwise() %>%
mutate(pred = names(.)[which.max(c_across(everything()))]) %>%
mutate(pred=factor(pred)) %>%
ungroup()

```

Reorder factor or predictions because R needs it even when the values are the same:

```

final_predictions$pred <- factor(final_predictions$pred, levels = levels(test_data$position_1))
confusionMatrix(final_predictions$pred, test_data$position_1)

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction  C PF PG SF SG
##          C  15  4  0  0  0
##          PF  6 15  0  3  0
##          PG  0  0 17  1  4
##          SF  0  0  0 13  4
##          SG  0  3  4  4 14
##
## Overall Statistics
##
##              Accuracy : 0.6916
##              95% CI : (0.595, 0.7773)
##      No Information Rate : 0.2056
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.6142
##
##      McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: C Class: PF Class: PG Class: SF Class: SG
## Sensitivity          0.7143    0.6818    0.8095    0.6190    0.6364
## Specificity          0.9535    0.8941    0.9419    0.9535    0.8706
## Pos Pred Value       0.7895    0.6250    0.7727    0.7647    0.5600
## Neg Pred Value       0.9318    0.9157    0.9529    0.9111    0.9024
## Prevalence           0.1963    0.2056    0.1963    0.1963    0.2056
## Detection Rate       0.1402    0.1402    0.1589    0.1215    0.1308
## Detection Prevalence 0.1776    0.2243    0.2056    0.1589    0.2336
## Balanced Accuracy     0.8339    0.7880    0.8757    0.7863    0.7535

```

It looks like that thresholding by youden index did not do any change at all.